PJT 1A

Benoit GADRET
Gaspard BEAUDIC

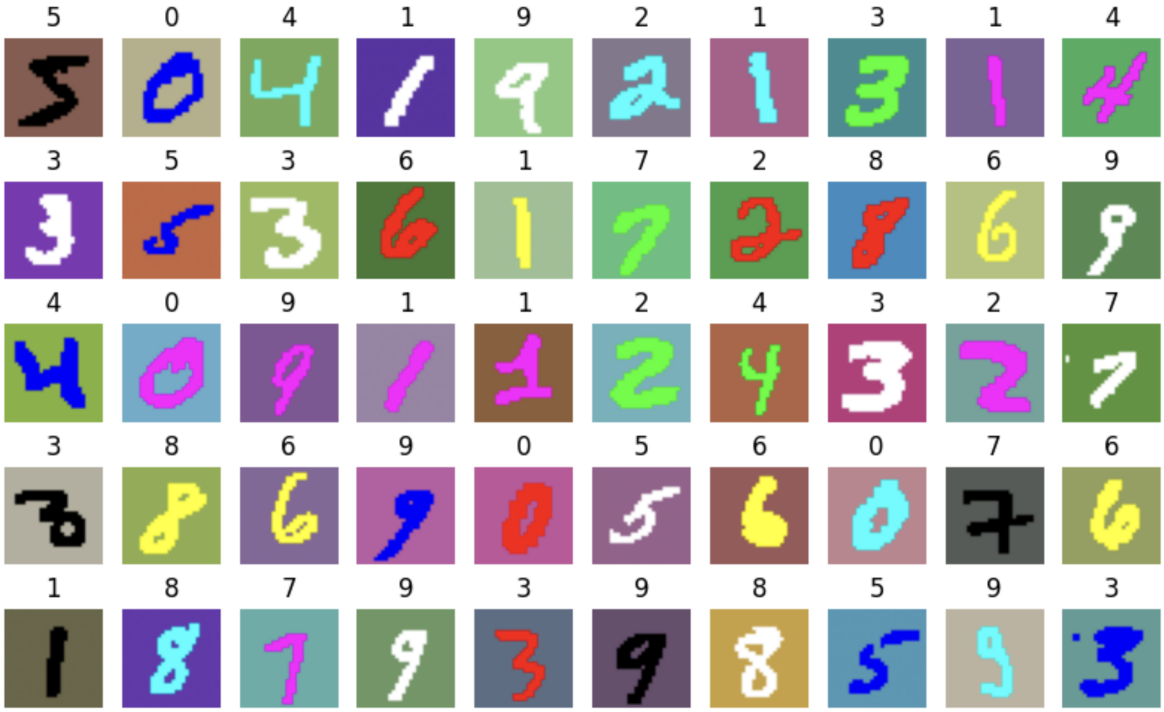


FIGURE 1 – Bibliothèque MINST RGB

Date de rendu : 12 janvier 2026

I Introduction

I.1 Objectifs du projet

Le but principal de ce projet est de résoudre un problème de **classification supervisée d'images** en utilisant des **réseaux de neurones convolutifs** (CNN), à partir d'un jeu de données spécifique (MNIST RGB).

Ce travail s'articule autour de plusieurs axes d'apprentissage :

1. **Découverte théorique :** Distinction entre l'apprentissage supervisé et non supervisé.
2. **Maîtrise des outils :** utilisant TensorFlow/Keras pour l'architecture du réseau, NumPy pour le calcul matriciel, et Matplotlib pour la visualisation
3. **Méthodologie technique :**
   — Construction de l'architecture du réseau.
   — Entraînement sur des bases de données massives.
   — Évaluation rigoureuse des performances (matrices de confusion, courbes de perte).
4. **Sensibilisation industrielle :** Compréhension des enjeux de déploiement de l'IA en milieu de production.

Table 1 – Bibliothèques principales utilisées

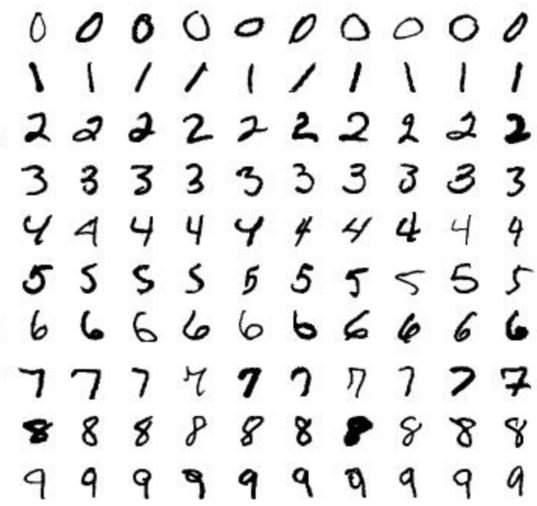
I.2 Le Dataset MNIST

Nous utilisons la base de données MINST, composée de miliers d'images (70 000) représentants des chiffres manuscrits (0 à 9). La taille de cette image est de 28×28. Initialement en niveaux de gris, le dataset a été complexifié en une version RGB (rouve vert bleu). Elle a été construite en générant des couleurs de chiffres et fonds aléatoires, ce qui va forcer le réseau de neuronnes à se concentrer sur la forme des items. Pour être exploitables par la bibliotheque Keras, nous devons effecturer des modiffications sur le format des images.

— conversion des images en tenseurs de dimension $(28, 28, 3)$

— les étiquettes sont transformées en vecteurs binaires via `to_categorical`.
Par exemple, la classe 3 est encodée comme :

$$3 \longrightarrow (0,\ 0,\ 0,\ 1,\ 0,\ 0,\ 0,\ 0,\ 0,\ 0)$$



MNIST en niveaux de gris

MNIST RGB

Table 2 – Dataset Minst

II Architecture du Réseau de Neurones

Layer (type)	Output Shape	Param #
conv2d_20 (Conv2D)	(None, 24, 24, 16)	1,216
max_pooling2d_16 (MaxPooling2D)	(None, 12, 12, 16)	0
conv2d_21 (Conv2D)	(None, 10, 10, 32)	4,640
max_pooling2d_17 (MaxPooling2D)	(None, 5, 5, 32)	0
conv2d_22 (Conv2D)	(None, 3, 3, 32)	9,248
flatten_12 (Flatten)	(None, 288)	0
couche_1 (Dense)	(None, 100)	28,900
couche_2 (Dense)	(None, 40)	4,040
dense_13 (Dense)	(None, 10)	410

Figure 2 – Architecture du réseau

II.1 Couche de Convolution

Des filtres balaient l'image pour y extraire des caractéristiques. Cette étape se fait grâce à la fonction d'activation ReLU (rectified linear unit), qui remplace toutes les valeurs négatives détectées en 0.

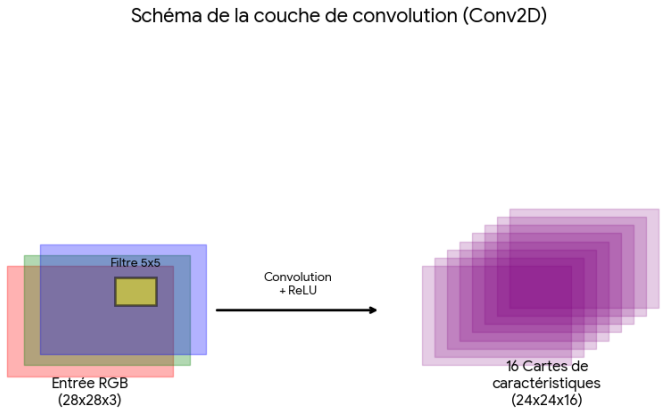


FIGURE 3 – Premiere couche de convolution du problème

La ligne de code est utilisée est la suivante :
`model.add(Conv2D(16, (5,5), activation='relu', input_shape=(28,28,3)))` Le processus se décompose de la sorte :

1. L'entrée est une image RGB de dimension $28 \times 28 \times 3$
2. On utilise 16 filtres (Kernels) de dimension $5 \times 5 \times 3$.
3. Utilisation de la fonction ReLU.
4. On obtient en sortie 16 éléments de dimension 24×24.

II.2 la couche `MaxPooling2D`

`model.add(MaxPooling2D(pool_size=(2,2)))` Cette fonction permet de réduire la taille des données en ne conservant que la valeur maximale d'un carré de données 2×2. Cela permet de diminuer par 4 le nombre de paramètres.

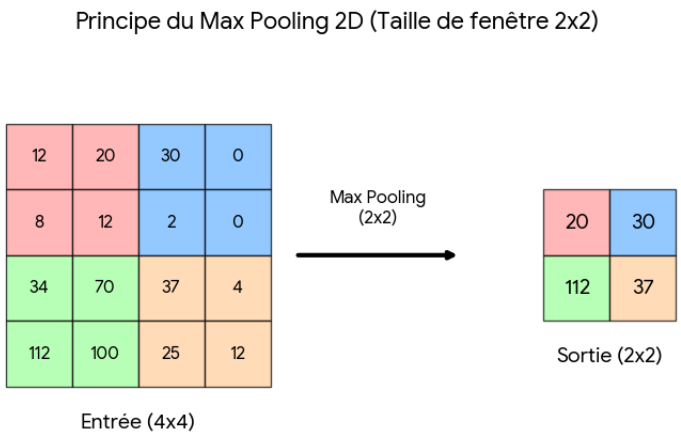


FIGURE 4 – MaxPooling2D

II.3 la couche `Flatten`

Cette couche flatten permet de "aplatir" les données d'entrée en un long vecteur 1D. Cela permet de faire une transition entre couches convolutives et denses.

II.4 la couche `Dense`

A l'aide des couches denses, nous créeons les différenctes couches du réseau de neuronnes, ou ces derniers sont tous conectés entre eux. La fonction Softmax permet de transformer les scores en sortie en probablilité.

$$\sum_{i=1}^{n} p_i = 1$$

Notre réseau s'articule de la maniere suivante :

- **Entrée Flatten :** 288 neurones.
- **Couche 1 :** 100 neurones avec fonction *ReLU*.
- **Couche 2 :** 40 neurones avec fonction *ReLU*.
- **Sortie :** 10 neurones avec fonction *Softmax*.

II.5 Compilation et Optimisation

Pour entrainer ce réseau, nous avons besoin de :

- **L'optimiseur Adam**, un algorithme qui réalise une descente de gradient stochastique.
- **La fonction Categorical Crossentropy**, qui mesure la différence entre la prédiction et la réalité.

III Analyse des résultats

III.1 Performances Globales et Précision

Nous avons évalué le modèle sur 10 000 images de test :

— Précision (Accuracy) : Le réseau atteint un taux de réussite de 98,13

— Perte (Loss) : La valeur de la fonction de perte finale s'établit à 0,0749.

III.2 Analyse par la Matrice de Confusion

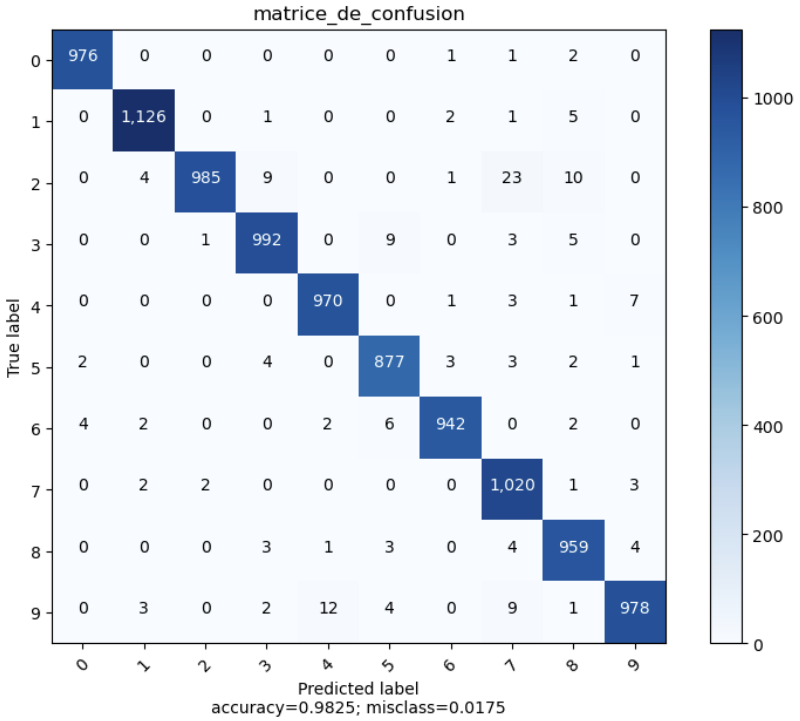


FIGURE 5 – Matrice de confusion

Pour visualiser ces résultats, nous avons généré une matrice de confusion, qui croise les classes réelles des prédictions. Nous en tirons les conclusions suivantes :

1. **Diagonale dominante :** la diagonale est très dense, le modèle est donc fiable pour toutes les classes.
2. **Ambiguités majeures :** nous remarquons tout de même des erreurs dans notre modele qui se concentrent sur des chiffres légerment similaires : (3 et 8, 4 et 9, 7 et 2 par exemple).

La fonction `ShowBadPredictions` nous permet d'aficher les confusions. Nous observons que dans certains cas les images étaient tracées de manière trop inhabituelle rendant leur identification difficile (même pour nous). De plus dans certains cas, le couleur choisie aléatoirement pour le chiffre est trop proche de celle du fond, complexifiant encore une fois l'identification.

IV Étude N°4 : Réseau Binaire Pair vs Impair

Dans cette partie, nous passons dun problème à 10 classes (10 classes : de 0 à 9) en un problème binaire (2 classes : Pair ou Impair). Nous allons nous concentrer sur 3 approches différente.

IV.1 Approche 1 : Transformation mathématique des prédictions

Dans cette première approche, nous utilisons le premier model (celui à 10 classes) sans le modifier.

- **Principe technique :** on applique l'opérateur modulo 2 (x mod 2) :
  - sur les labels : transformation des étiquettes de la base de données de 0 à 9 en 0 (pair) ou 1 (impair). Par exemple, le label « 7 » devient « 1 » ;
  - sur les prédictions : transformation du chiffre prédit par le modèle (celui prévu par l'**argmax**) de la même manière.
- **Nouvelle comparaison :** à présent, au lieu de vérifier si « 7 = 7 », le système vérifie si « 1 = 1 ».

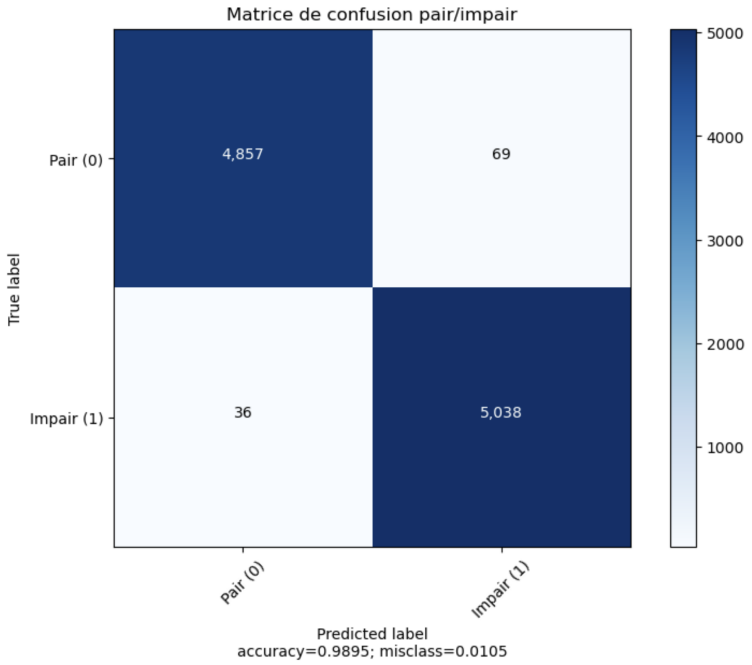


FIGURE 6 – Matrice de confusion

En tracant la matrice de confusion, nous nous rendons compte que l'accuracy est légèrement supérieure au model à 10 classes. Nous sommes passés de 0,9825 à 0,9895, car la nature même de l'erreur a été changé dans ce nouveau cas. En effet, dans le model initial confondre deux nombre représentait une erreur même s'ils avaient la même parité. Prenons un exemple pour expliquer ce qui suit : si le réseau confondait un "3" avec un "5", c'était compté comme une erreur. Cependant, après le modulo, comme le "3" et le

"5" sont tous deux impairs, cette erreur de confusion disparaît totalement du calcul final. Ce sont ces cas qui font que l'accuracy a augmenté d'un modele à l'autre.

IV.2 Approche 2 : Conception d'un Réseau Binaire Dédié

IV.2.1 Objectif de l'approche

L'objectif de cette seconde approche est de vérifier l'hypothèse de la **spécialisation**. Contrairement à l'approche précédente, qui réinterprétait les résultats du réseau à 10 classes (du début) pour reconnaître les 10 chiffres, nous cherchons ici à entraîner un réseau de neurones à deux classes : Pair (0) et Impair (1).

L'intérêt de cette approche est de déterminer si un modèle dédié à la reconnaissance pair/impair, avec des poids optimisés pour cette tâche binaire, offre une meilleure précision ou une convergence plus rapide que le modèle précédent (à 10 classes).

IV.2.2 Préparation des données (Data Engineering)

Avant l'entraînement, une transformation des étiquettes (*labels*) est nécessaire pour passer d'un problème à 10 classes à un problème à 2 classes.

- **Transformation mathématique :** Nous appliquons l'opérateur modulo (`% 2`) sur l'ensemble des labels d'entraînement et de test.
  - Les chiffres 0, 2, 4, 6, 8 sont réétiquetés **0**.
  - Les chiffres 1, 3, 5, 7, 9 sont réétiquetés **1**.
- **Encodage :** Pour correspondre à la couche de sortie du réseau, ces étiquettes scalaires sont converties en vecteurs binaires via la fonction `to_categorical`.
  - Classe 0 (Pair) $\rightarrow [1, 0]$.
  - Classe 1 (Impair) $\rightarrow [0, 1]$.

IV.2.3 Architecture du Modèle

Le modèle construit garde une architecture de type CNN (*Convolutional Neural Network*) comme l'approche précédente. En effet, la nature des données nécessite une extraction des caractéristiques comme pour le modèle du tronc commun.

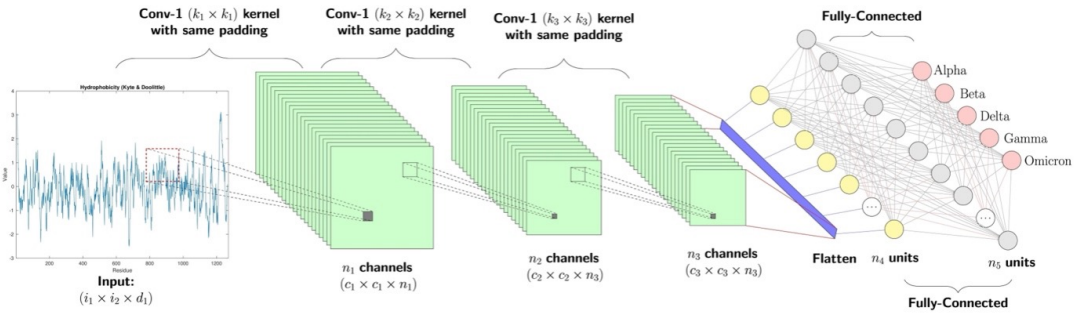


FIGURE 7 – Schéma de principe d'un CNN

Cependant, l'architecture a été adaptée pour la classification binaire :

- **Couches de Convolution (Feature Extraction) :** Identiques au modèle précédent (alternance de `Conv2D` et `MaxPooling2D`).

— **Couche de Sortie (Output Layer) :** C'est la modification majeure. La dernière couche `Dense` comporte désormais **2 neurones** (au lieu de 10) avec une fonction d'activation **Softmax**.

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 24, 24, 16)	1,216
max_pooling2d_4 (MaxPooling2D)	(None, 12, 12, 16)	0
conv2d_7 (Conv2D)	(None, 10, 10, 32)	4,640
max_pooling2d_5 (MaxPooling2D)	(None, 5, 5, 32)	0
conv2d_8 (Conv2D)	(None, 3, 3, 32)	9,248
flatten_2 (Flatten)	(None, 288)	0
couche_1 (Dense)	(None, 100)	28,900
couche_2 (Dense)	(None, 40)	4,040
dense_3 (Dense)	(None, 10)	410
dense_4 (Dense)	(None, 2)	22

Total params: 48,476 (189.36 KB)

Trainable params: 48,476 (189.36 KB)

Non-trainable params: 0 (0.00 B)

FIGURE 8 – Résumé de l'architecture du modèle binaire (model.summary())

IV.2.4 Analyse des Résultats

Sur le jeu de test, ce modèle spécialisé atteint une précision (*accuracy*) de **99.94%**. Ce résultat est sensiblement le même que celui de l'approche 1 (98,95%) qui utilisait le réseau à 10 classes.

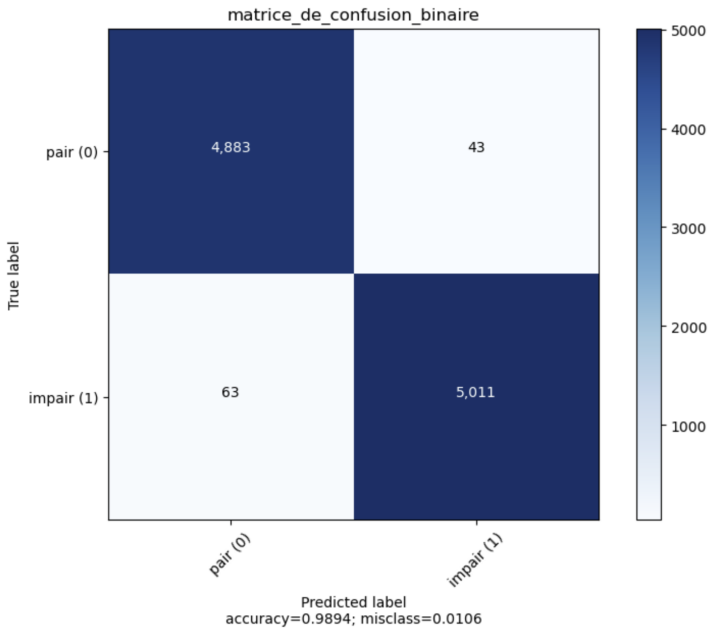


FIGURE 9 – Matrice de confusion du modèle binaire

Ainsi, cela montre que malgré le fait que les caractéristiques visuelles des chiffres soient

les mêmes, les dernières couches denses spécialisées pour ce problème permettent d'avoir une très bonne précision lors de la distinction pair et impair.

Limite : C'est la méthode la plus coûteuse en ressources. Cette approche oblige le réseau à réapprendre "depuis zéro" comment détecter des bords, des courbes et des formes, ce qui peut être long et redondant.

IV.3 Approche 3 : Transfert d'Apprentissage (Transfer Learning)

L'idée principale de cette partie est de réutiliser une partie du premier model à 10 classes pour ne pas avoir à ré-entraîner toute la partie convolutionnelle (très coûteuse en temps).

IV.3.1 Extraction de caractéristiques (Feature Extraction)

Nous avons tronqué le premier modèle au niveau de la couche `Dense(100)` (nommée `couche_1`).

- **Entrée :** Image (28, 28, 3).
- **Sortie :** Vecteur de 100 nombres (les caractéristiques).

```
# Creation du modele d'extraction
NewModel = Model(inputs=model.inputs,
    outputs=model.layers[-3].output)
# Extraction des features (transformation des images en vecteurs)
Xtrain_Petitmodel = NewModel.predict(XtrainRGB)
Xtest_Petit_model = NewModel.predict(XtestRGB)
```

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 28, 28, 3)	0
conv2d_9 (Conv2D)	(None, 24, 24, 16)	1,216
max_pooling2d_6 (MaxPooling2D)	(None, 12, 12, 16)	0
conv2d_10 (Conv2D)	(None, 10, 10, 32)	4,640
max_pooling2d_7 (MaxPooling2D)	(None, 5, 5, 32)	0
conv2d_11 (Conv2D)	(None, 3, 3, 32)	9,248
flatten_3 (Flatten)	(None, 288)	0
couche_1 (Dense)	(None, 100)	28,900

FIGURE 10 – Modèle tronqué

IV.3.2 création du "Petit Modèle"

L'étape suivante consiste à créer un nouveau réseau de neuronnes qui prend en entrée le vecteur de taille 100
le code utilisé pour sa création est le suivant :

```
Petit_model = Sequential()
Petit_model.add(Dense(50, activation='relu', input_shape=(100,)))
Petit_model.add(Dense(2, activation='softmax'))
```

L'architecture est la suivante :

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 50)	5,050
dense_13 (Dense)	(None, 2)	102

FIGURE 11 – petit model

IV.3.3 Analyse technique et performances

L'efficacité de cette approche repose sur le principe de **Transfert d'Apprentissage**. Utiliser le vecteur de sortie de la `couche_1` fait que l'on traite avec ce "petit model" des informations très pertinentes à propose de la forme du chiffre.

L'entraînement a été réalisé sur 15 époques. Nos observations sont les suivantes :

— **Rapidité d'exécution :** Le modèle étant très léger (seulement 5152 paramètres), les calculs sont donc plus rapides.

— **Précision Record (Accuracy) :** Le modèle atteint une accuracy de **98,89 %**.

— **Analyse de la Matrice de Confusion :** elle confirme que l'augmentation de l'accuracy (par rapport au problème à 1à classes) provient du passage de 10 à 2 classes. Le système n'est plus pénalisé par les erreurs entre chiffres de même parité (comme la confusion entre un 3 et un 5). Le modèle commet donc moins d'erreurs, il est plus performant (et le problème plus simple). Ceci a été déjà observé pour l'**Approche 1**.

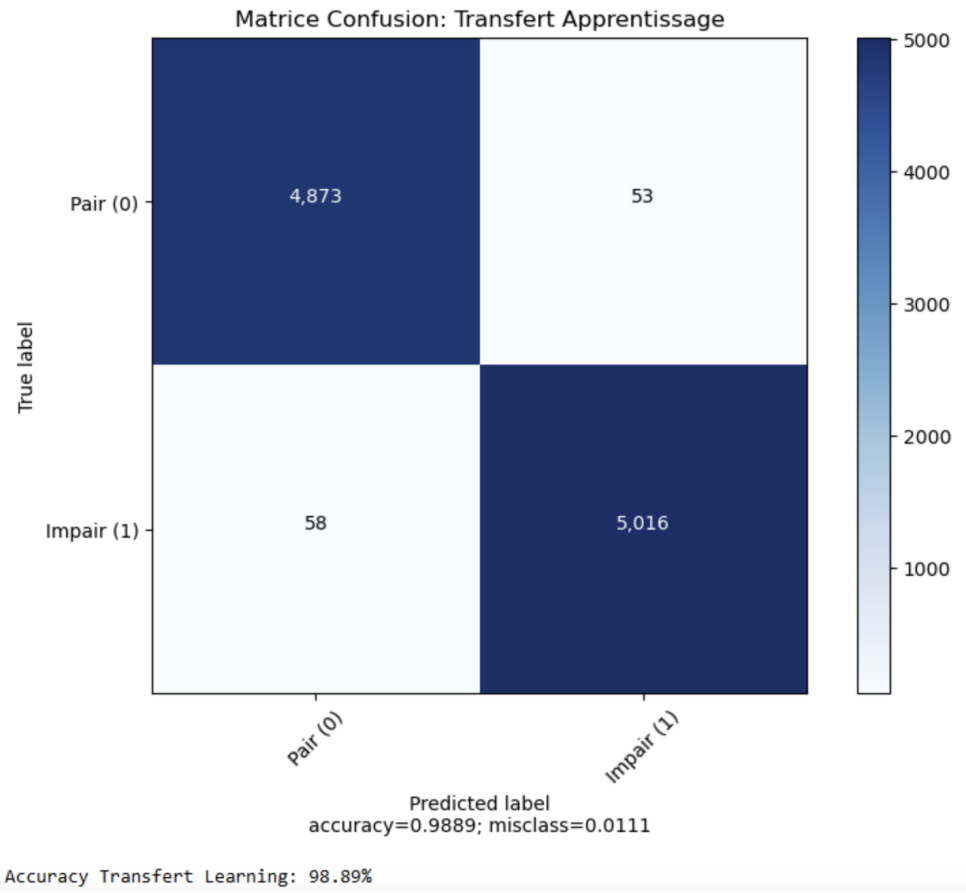


FIGURE 12 – Matrice de confusion

Nous retenons donc de l'approche par transfert d'apprentissage que cette méthode garde la puisssance que l'on avait avec le CNN classique (développé dans le problème à 10 classes) tout en économisant enormément de calculs, grâce à la simplicité de l'architecture du "petit model". Cest donc la meilleure option, qui offre une tres bonne accuracy tout en économisant des calculs

IV.3.4 Pour aller plus loin

Contrairement à la première version du transfert d'apprentissage, dans la **question 7**, nous avons tronqué le modèle une couche en dessous, passant de 100 à 40 valeurs : cela permet d'encore plus alléger le calcul tout en améliorant la précision (99%)

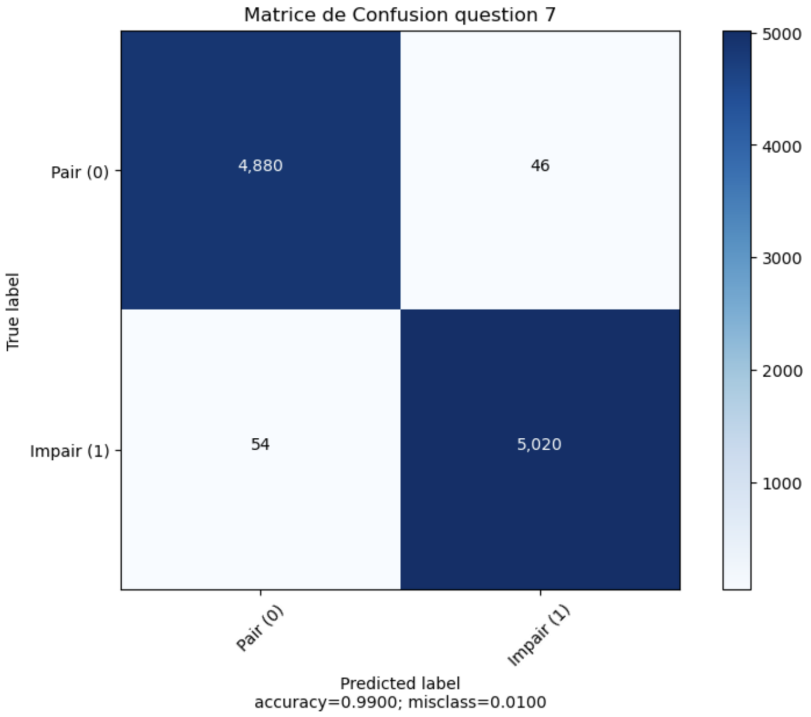


FIGURE 13 – Matrice de confusion de la question 7

V Conclusion

Ce projet nous a permis de nous familiariser avec la notion de réseau de neuronnes (CNN) sur `Python`. En partant du dataset MINST en niveaux de gris puis complexifié en une version RGB, nous avons réussi à extraire les caractéristiques de ces images avec un problème à 10 classes. La précision de ce premier travail était de 98,25 %

Ensuite, notre étude était une classification binaire : elle consistait à déterminer si un nombre était pair ou impair. Ceci a été mené en deux temps : nous avons d'abord appliqué une opération de modulo sur les prédictions issues du modèle initial à 10 classes, avant de réaliser cette même opération sur les étiquettes (labels) de référence pour permettre la comparaison.
Nous remarquons alors qu'en réduisant l'espace de décision, des erreurs sont absorbées (les confusions entre chiffres de même parité ne sont plus pénalisées), ce qui augmente indéniablement la précision.

Enfin, la dernière approche concernant le Transfert d'Apprentissage s'est révélée être la stratégie la plus performante. En réutilisant les couches de convolution du premier réseau pour alimenter un modèle binaire extrêmement simple ("petit model") de seulement 5 152 paramètres, nous avons atteint une très bonne précision tout en réduisant le temps de calcul. Il est donc plus pertinent d'utiliser un "apprentissage" déjà réalisé que d'entrainer un model complet.